Learning Domain-Independent Heuristics for Classical Planning with Graph Neural Networks

1. One Line Summary

This paper proposes three graph representations of planning problems (SLG, FLG, and LLG) and shows that Graph Neural Networks can learn effective domain-independent planning heuristics from these representations, with lifted graphs providing the best practical scalability and performance.

2. Problem

Classical planners rely heavily on heuristic functions to estimate the distance from a state to the goal. Strong heuristics such as hFF and LM-cut often require significant domain knowledge and computation. Recent learning-based approaches attempt to replace these heuristics with neural networks, but existing graph representations suffer from two major limitations:

1. **Loss of planning information**, such as delete effects.
2. **Grounding explosion**, where all possible action instances must be generated before planning.

The paper investigates whether better graph representations can enable Graph Neural Networks (GNNs) to learn more accurate and scalable planning heuristics.

3. Core Idea

The central idea is to convert a planning problem into a graph and allow a Message Passing Neural Network (MPNN) to learn heuristic values directly from graph structure.

The authors propose three graph representations:
SLG (STRIPS Learning Graph)
Represents:

- Propositions
- Actions
- Preconditions
- Add effects
- Delete effects

SLG improves upon previous graph representations by explicitly preserving delete effects.

FLG (FDR Learning Graph)
Built from Finite Domain Representation (FDR).
Represents:

- Variables
- Variable values
- Actions

FLG preserves variable value relationships that are hidden in STRIPS.

LLG (Lifted Learning Graph)
Represents:

- Objects
- Predicate schemas
- Action schemas
- State and goal facts
  while avoiding grounding of actions.

The key insight is that preserving lifted structure dramatically reduces graph size and improves generalization.

4. Key Concepts

Classical Planning
A planning problem is represented as:
$\Pi = \langle S, A, s_0, G \rangle$

The objective is to find a sequence of actions transforming s_0 into a goal state.

STRIPS
Represents states as sets of propositions.
An action contains: <pre,add,del>
Example:
Action: Stack(A,B)
Preconditions:

- Holding(A)
- Clear(B)
  Add:
- On(A,B)
  Delete:
- Holding(A)

- Clear(B)

FDR (Finite Domain Representation)
Instead of storing many independent propositions, FDR stores variables with finite domains.
Example:
loc(A) $\in$ {Table, B, C}
This naturally captures mutual exclusivity and introduces additional structure that can be exploited by learning algorithms.

Lifted Planning
Instead of grounding:
stack(A,B)
lifted planning keeps:
stack(x,y)

This avoids generating all possible action instances and improves scalability.

Graph Neural Networks

GNNs perform message passing between connected nodes.

Each layer updates node representations using information from neighboring nodes, allowing the network to learn relationships between facts, actions, predicates, and goals.

5. Method Pipeline

The proposed planner, GOOSE (Graphs Optimised for Search Evaluation), works as follows:

Current Search State
→ Graph Construction (SLG / FLG / LLG)
→ MPNN
→ Predicted Heuristic
→ Greedy Best-First Search

Training data is generated from optimal plans.

For a plan of length d:

State s_0 receives label d

State s_1 receives label d−1

...

Goal state receives label 0

Thus a single optimal plan generates many supervised training examples.

6. Results

Theoretical Results

- MPNNs on SLG and FLG can represent hmax and hadd.
- SLG and FLG are strictly more expressive than STRIPS-HGN.
- LLG cannot perfectly represent hmax or hadd.
- No graph representation can generally learn h+ or h*.
- No universal approximation guarantee exists for h*.

These results establish a trade-off between expressiveness and scalability.

Experimental Results

Domain-Dependent Training

LLG performs best in most domains.

Reason:

LLG explicitly preserves predicate-level information, making it easier to learn domain-specific patterns.

Example:

Instead of seeing thousands of grounded facts, the network directly sees predicate schemas such as:

on(x,y)

which often correlate strongly with heuristic values.

Domain-Independent Training

SLG achieves the strongest generalization.

Reason:

Its simpler representation reduces overfitting and encourages learning more generic planning behavior.

FLG

Although theoretically expressive, FLG tends to overfit because it encodes additional planning structure.

Sokoban

Performance deteriorates significantly.

This likely reflects the difficulty of capturing long-range dependencies and deadlocks using standard message-passing GNNs.

7. Weaknesses

- LLG improves scalability but sacrifices expressiveness.Theoretical analysis proves that some planning tasks become indistinguishable in LLG.
- Although grounded actions are removed, grounded state and goal facts remain. Therefore grounding explosion is reduced rather than eliminated.
- Message passing is fundamentally local.Long reasoning chains may require many propagation steps, making it difficult to capture deep planning dependencies.
- Domains such as Sokoban expose limitations of local graph reasoning and heuristic approximation.

8. My Thoughts

- Is GNNs a better option for planning problems than transformer ?? why so what makes gnns better
- Why do GNNs have greater generalization capability. Even transformers do provide output for larger ones right?
- for representation why are we complicated so much.... so like we just have nodes = obj and edges to denote any sort of relation between them like if a action between the 2 then something if its a predicate then something.
